Free-XOR Garbling Is Not Binding Against the Garbler

What the On-Chain Label Reveal Buys in BitVM-Style Protocols

Aria Naraghi*

August 2026

Preprint. Comments welcome.

Abstract

BitVM-style protocols move SNARK verification off the Bitcoin chain by having a prover garble a verifier circuit and having challengers evaluate it. Soundness rests on the prover being unable to make an honest evaluation accept without a valid witness. We observe that no standard garbling security property gives this: correctness, privacy and authenticity are all stated against adversaries who do *not* hold the encoding information, and in this setting the adversary is the garbler, who holds all of it. Every deployed and proposed design closes the gap the same way, by revealing the garbled input labels on-chain, which is why on-chain cost is linear in the size of the proof.

We isolate the missing property, which we call *evaluation-binding*, and show it is not free. Under the native decoding of the half-gates scheme it fails in two trials. Under BitVM3-core's decoding commitment it is unobservable, because only the false output label is committed. Stated in the natural way it is false for a trivial reason. Repaired, its security is at most $2^{k/2}$ rather than 2^k whenever the circuit admits an affine split reachable from the output, which free-XOR makes structurally likely: we give a working birthday attack, measured at scaling exponent 1.02 over $k \in [24, 44]$, that produces label vectors that encode no input yet decode to TRUE.

We then ask whether the circuit actually deployed has the structure. Using a custom evaluation mode over the BN254 Groth16 verifier of the BitVM Alliance implementation (1.03×10^{10} gates, 2.71×10^9 non-free, 3302 input wires), we show the output's linear frontier is a single atom and that none of the 638,658 non-free gates in the circuit's tail has separably controllable operands. The deployed system is therefore *not* vulnerable, but it is safe by accident: the property that saves it is a consequence of Groth16's final equality check, is stated nowhere, and is not obviously preserved under a change of proof system. We contribute the definition, the attacks, an exploit, and a tool that decides the structural condition.

1 Introduction

Bitcoin cannot verify a SNARK. A Groth16 verifier [6] compiles to roughly a gigabyte of Bitcoin Script and a block holds four megabytes. The BitVM line of work [11, 12, 14] answers this optimistically: do not verify on-chain, but arrange that an incorrect claim can be *disproved* on-chain cheaply.

BitVM3 [14] realises the fraud proof with garbled circuits. An operator garbles the SNARK verifier once, publishes the garbled circuit off-chain, and posts a transaction whose witness reveals the garbled encoding of its claimed proof. Any party may evaluate the circuit on that

*Independent. Correspondence: `arianari97@gmail.com`. Artifacts: `https://github.com/<user>/garbler-binding`

encoding; if the proof is invalid the evaluation yields a designated label L^*, and revealing L^* on-chain spends the operator's deposit. The reported cost is roughly \$9 total with a fraud proof under \$0.20, a thousandfold improvement on BitVM2.

The on-chain reveal is expensive and it is the reason on-chain cost is $O(|\pi|)$ in the proof size. Three separate lines of recent work exist to manage its consequences. Eagen's Glock [4] identifies why it works at all, observing that projective signatures compose with garbled input encodings in what he calls an "auspicious harmony," and then makes the identification literal by *defining* input labels to be signature shares. Duty-Free Bits [8] exists because that identification forces projectivity: its authors state plainly that projectivity is mandated by the on-chain authentication mechanism. Mosaic [9] reduces the on-chain footprint so it no longer scales with the number of cut-and-choose copies, and opens by noting that the verifier needs the prover's input labels, which the prover must post on chain.

The gap

What does the on-chain reveal actually buy, cryptographically?

BitVM3's soundness argument answers this precisely, in its own words: a malicious prover posting the assertion reveals, *by extractability* of the signature scheme, the encoding $\mathsf{En}(e, \pi)$; and then *by correctness* of the garbling scheme, evaluating on that encoding reproduces the verifier's verdict. The second step depends on the first. Correctness of a garbling scheme is a statement about valid encodings and says nothing about $\mathsf{Ev}(F, L)$ for L outside the encoding space. Extractability is what supplies the hypothesis that correctness requires.

So the on-chain reveal purchases exactly one thing: the guarantee that the labels the evaluator acts on are well-formed. Remove it and the soundness argument does not weaken, it loses its first line.

One might hope authenticity covers the residue. It does not, and the definition says so. In the formulation of Zahur, Rosulek and Evans [15], the authenticity adversary is given (F, X) *alone*: not the encoding information e, not the decoding information d, not the global offset R. The garbler holds all three. Authenticity is silent about the garbler by construction, and correctly so, since it was formulated for a different threat model [2].

Contributions

1. **A definition.** We isolate *evaluation-binding* (Definition 1): no PPT adversary holding (F, e, d) can produce an input label vector on which honest evaluation decodes to True unless the circuit is satisfiable. This is the property BitVM-style protocols need and obtain, today, only through the on-chain reveal. It is not implied by correctness, privacy or authenticity.

2. **Five attacks** (Section 4). Native half-gates decoding admits forgery in two expected trials. BitVM3-core commits to the false output label only, making success unobservable off-chain. The encoding-form statement of the property is false for a trivial free-XOR reason. Even fully repaired, the property has a $2^{k/2}$ ceiling. And the existing soundness proof admits no partial repair.

3. **A working exploit** (Section 5). A faithful, k-parameterised implementation of the half-gates scheme, validated exhaustively against plaintext evaluation, in which the birthday attack produces label vectors that are the encoding of *no* input yet decode to True under committed decoding. Measured scaling exponent 1.02 against a birthday prediction of 1.00, over $k \in [24, 44]$; speedup over naive search of $3259\times$ already at $k = 28$.

4. **A measurement of the deployed circuit** (Section 6). A custom evaluation mode over the BitVM Alliance BN254 Groth16 verifier [3] establishes that its output's linear frontier is a single atom and that none of the 638,658 non-free gates in its tail has separably controllable operands. The attack does not apply to it.

5. **A design requirement.** The deployed circuit is safe because Groth16 ends in an equality check that aggregates the whole pairing result. Nobody chose that. We argue evaluation-binding should become an explicit, checked requirement, and we release the tool that checks it.

We emphasise what this paper does *not* claim. We do not break any deployed system. The deployed system is protected by the on-chain reveal, which makes evaluation-binding moot; and even without it, we measured the circuit and it resists our attack. The claim is that the guarantee is unstated, the margin is smaller than the parameter suggests, and the protection is incidental.

2 Preliminaries

2.1 Garbling schemes

A garbling scheme [2] is a tuple $(\mathsf{Gb}, \mathsf{En}, \mathsf{Ev}, \mathsf{De})$. On a circuit F, $\mathsf{Gb}(1^k, F)$ outputs a garbled circuit $\hat{F}$, encoding information e and decoding information d. $\mathsf{En}(e, x)$ maps a plaintext input to garbled input labels, $\mathsf{Ev}(\hat{F}, X)$ evaluates, and $\mathsf{De}(d, Y)$ decodes. We use the three standard properties: *correctness* $(\mathsf{De}(d, \mathsf{Ev}(\hat{F}, \mathsf{En}(e, x))) = F(x))$, *privacy*, and *authenticity*.

Authenticity, verbatim in structure from [15]: given $(\hat{F}, X)$ alone, no adversary can produce $\tilde{Y} \neq \mathsf{Ev}(\hat{F}, X)$ with $\mathsf{De}(d, \tilde{Y}) \neq \perp$ except with negligible probability. The quantifier is the point. The adversary receives the garbled circuit and one encoded input, and nothing else.

2.2 Half-gates with free-XOR

We work with the half-gates scheme of [15] with the free-XOR optimisation of [10], which is what the BitVM Alliance implementation uses. Writing H for a circular-correlation-robust hash [7]:

$$
\begin{aligned}
\mathsf{Gb}: \quad & R \leftarrow\$ \{0,1\}^{k-1}1, \qquad W_i^0 \leftarrow\$ \{0,1\}^k \text{ for inputs}, \qquad W_i^1 = W_i^0 \oplus R \\
& \text{XOR gate: } W_i^0 = W_a^0 \oplus W_b^0 \qquad \text{(no ciphertexts)} \\
& \text{AND gate: } (W_i^0, T_{Gi}, T_{Ei}) \leftarrow \mathsf{GbAnd}(W_a^0, W_b^0) \\
& \text{output: } d_i = \mathsf{lsb}(W_i^0) \\
\mathsf{En}: \quad & X_i = e_i \oplus x_i R \\
\mathsf{Ev}: \quad & \text{XOR: } W_i = W_a \oplus W_b \\
& \text{AND: } s_a = \mathsf{lsb}(W_a),\ s_b = \mathsf{lsb}(W_b), \\
& \qquad W_{Gi} = H(W_a, j) \oplus s_a T_{Gi}, \qquad W_{Ei} = H(W_b, j') \oplus s_b(T_{Ei} \oplus W_a), \\
& \qquad W_i = W_{Gi} \oplus W_{Ei} \\
\mathsf{De}: \quad & y_i = d_i \oplus \mathsf{lsb}(Y_i)
\end{aligned}
$$

Two structural facts drive everything below. First, every wire's two labels differ by the same global offset R, so any gate combining labels by XOR is $\mathbb{F}_2$-affine in them. Second, the scheme as published provides correctness and privacy but, in the authors' own statement, does not provide authenticity; authenticity requires a modified decoding procedure.

2.3 BitVM3-core

BitVM3-core [14] instantiates the above with $k = 128$. The prover garbles the SNARK verifier, publishes $\hat{F}$ and d off-chain, and posts an Assert transaction whose witness carries one 65-byte Schnorr adaptor signature per 8-bit digit of the proof. By extractability of that signature

scheme, any observer recovers $\mathsf{En}(e, \pi)$ from the chain. Writing L^* for the output label decoding to FALSE, the Assert output carries a hash lock on $H(L^*)$; a challenger who obtains L^* by evaluation spends it and takes the deposit.

Note that $H(Z^{\text{TRUE}})$ is never published. In the original design it need not be: the operator wins by the *absence* of a disproof, so no party ever has to recognise success.

3 Evaluation-binding

Definition 1 (Evaluation-binding)**.** A garbling scheme $(\mathsf{Gb}, \mathsf{En}, \mathsf{Ev}, \mathsf{De})$ is *evaluation-binding* for a circuit F if for every PPT adversary $\mathcal{A}$ given $(\hat{F}, e, d) \leftarrow \mathsf{Gb}(1^k, F)$, the probability that $\mathcal{A}$ outputs a label vector L with $\mathsf{De}(d, \mathsf{Ev}(\hat{F}, L)) = \text{TRUE}$, when F is unsatisfiable, is negligible.

Two remarks on the shape of the definition.

First, it must be stated over *outcomes* and not over the form of L. The tempting statement, that L must equal $\mathsf{En}(e, x)$ for some satisfying x, is false; see Attack 3.

Second, the property is not new to this paper in substance. Glock [4] states its security goal as: the evaluator learns the output label *if and only if* the garbler authenticated an input on which the circuit evaluates to that output. The "only if" direction is evaluation-binding. Every existing construction discharges it by the on-chain reveal rather than by proving anything about the garbling scheme. What is new here is the observation that the obligation exists, and what happens when one tries to meet it directly.

Observation 1. Evaluation-binding is not implied by correctness, privacy or authenticity. Correctness quantifies only over valid encodings. Privacy is irrelevant, and indeed the schemes used in this setting are deliberately privacy-free. And the authenticity experiment withholds e, d and R from the adversary, whereas the garbler holds all three.

4 Attacks

Attack 1 (Native decoding admits trivial forgery)**.** Under the decoding of Section 2.2, $\mathsf{De}(d, Y) = d \oplus \mathsf{lsb}(Y)$ is a single XOR against one bit of the label. Every k-bit string decodes to a valid output bit; there is no $\perp$. A malicious garbler therefore samples arbitrary L, evaluates locally, and resamples if the result decodes to FALSE. The expected number of trials is two.

Our implementation forges in 1, 1 and 3 trials at $k = 32, 64, 128$ respectively. The repair is standard [15]: commit to both output labels, $d = (H(Z^0), H(Z^1))$, and return $\perp$ on a mismatch. All subsequent attacks assume this repair.

Attack 2 (One-sided decoding commitment)**.** BitVM3-core publishes $H(L^*)$ and not $H(Z^{\text{TRUE}})$. A party evaluating off-chain can therefore recognise failure and nothing else: TRUE and garbage are the same observation. Any protocol that asks a challenger to act on a successful evaluation is unimplementable as specified.

The repair is 32 bytes at setup. We flag it because it is invisible while the on-chain reveal is present, and becomes load-bearing the moment it is removed.

Attack 3 (Encoding-form binding is false)**.** Let S be any set of input wires whose parity into every application of H is even; two wires meeting at a common XOR gate suffice. For any $\rho \notin \{0, R\}$ set $L_m = W_m^0 \oplus \rho$ for $m \in S$ and $L_m = W_m^{x_m}$ otherwise. The ρ cancels at the XOR, no hash is ever applied to a perturbed label, and the evaluation is bit-identical to the honest one. Thus L is not $\mathsf{En}(e, x)$ for any x, yet decoding succeeds.

This does not help a cheating prover on its own. It matters because it fixes the shape of Definition 1, and because the mechanism it exposes, free-XOR rendering regions of the circuit $\mathbb{F}_2$-affine in the labels, is what the next attack exploits.

k	H evaluations	$2^{k/2}$	2^k	no valid encoding	decodes
24	20,722	4.10×10^3	1.68×10^7	yes	TRUE
28	105,924	1.64×10^4	2.68×10^8	yes	TRUE
32	280,360	6.55×10^4	4.29×10^9	yes	TRUE
36	1,170,344	2.62×10^5	6.87×10^{10}	yes	TRUE
40	4,349,874	1.05×10^6	1.10×10^{12}	yes	TRUE
44	24,420,688	4.19×10^6	1.76×10^{13}	yes	TRUE

Table 1: Attack 4, measured. "No valid encoding" is an exhaustive check that the forged vector equals $\mathsf{En}(e, x)$ for none of the 2^n inputs. Decoding uses the committed variant, so TRUE means the evaluation landed exactly on Z^{TRUE}.

k	birthday H evals	naive H evals	speedup
20	5,702	1,160,196	203×
24	20,722	36,204,000	1,747×
28	105,924	345,161,504	3,259×

Table 2: Head to head against unstructured search for the same target at the same k. The speedup itself grows as $2^{k/2}$.

Attack 4 (Birthday attack: the $2^{k/2}$ ceiling)**.** Suppose the output label can be written $Z = g(L_S)\oplus h(L_T)$ for disjoint input-wire sets S, T, which holds whenever the paths from S and from T merge only through XOR gates after their last application of H. Then: tabulate $g(L_S)$ over $2^{k/2}$ garbage assignments to L_S; sweep $2^{k/2}$ garbage assignments to L_T looking up $Z^{\text{TRUE}}\oplus h(L_T)$. A collision is expected and yields L with $\mathsf{Ev}(\hat{F}, L) = Z^{\text{TRUE}}$.

Modelling H as a random oracle, an unstructured search for the target costs 2^k. The split reduces it to $2^{k/2}$, and van Oorschot–Wiener parallel collision search [13] makes that memory-light and perfectly parallelisable. For the deployed parameter $k = 128$ the ceiling is 2^{64}.

At 10^9 hash evaluations per core-second, 2^{64} is roughly 585 core-years, or about three weeks on 10,000 cores. Whether that is alarming depends on the value secured; Clementine [1] has processed on the order of 150 BTC.

Attack 5 (No partial repair to the soundness proof)**.** BitVM3's soundness proof derives $L_\pi = \mathsf{En}(e, \pi)$ from extractability and only then applies correctness. Removing the on-chain reveal deletes the premise of the second step rather than weakening its conclusion. There is no intermediate position in which a weaker on-chain reveal yields a weaker but still sufficient guarantee.

5 Implementation

We implement the half-gates scheme of Section 2.2 in Rust, parameterised by label length k, following [15] Figure 2 line by line. We do not use the BitVM Alliance implementation [3] here because it fixes $k = 128$, placing Attack 4 beyond demonstration. Before any attack runs, the garbling is validated exhaustively against plaintext evaluation on every input of every test circuit at $k \in \{16, 32, 64, 128\}$.

The test circuit for Attack 4 is $\mathsf{out} = (a \wedge b) \oplus (c \wedge d)$: two AND gates and four inputs, the minimal circuit exhibiting the split. We hold L_b, L_d at honest labels and vary L_a, L_c over garbage.

Table 1 reports the attack, Table 2 the comparison against unstructured search. Fitting the measured work against k over $[24, 44]$ gives a scaling exponent of $\mathbf{1.02}$, against 1.00 predicted

input wires	3,302
gates executed	10,338,696,819
non-free gates	2,714,835,821
output sketch popcount	64/64
backward steps through free gates	1
frontier atoms (odd parity)	**1**
unresolved wires	0
top gate operand A / B popcount	64/64, 64/64
mutually exclusive residue classes	none
non-free gates in recorded tail	638,658
with separably controllable operands	**0**

Table 3: One full pass over the BN254 Groth16 verifier circuit.

by the birthday bound. Every row of Table 1 is a label vector that is the encoding of no input whatsoever and which an honest evaluator accepts as a proof of TRUE.

6 Does the deployed circuit have the structure?

Attack 4 needs a split. Section 5 demonstrates it on a circuit built to have one. This section asks whether the circuit BitVM3-core actually garbles has one.

6.1 Method

We implement a custom evaluation mode for the BitVM Alliance streaming circuit builder [3] and run the real `groth16_verify` circuit end to end. Each wire carries two pieces of information.

A dependency sketch. Input wire i is seeded with bit $i \bmod 64$ and every gate ORs its operands, so a wire's sketch records which of 64 residue classes of input wires it depends on.

A recorded tail. The last 2^{21} gates are kept with their wire identifiers and sketches, which lets the *linear frontier* of the output be reconstructed exactly offline: from the output wire, step back through free gates only, stopping at non-free gates or circuit inputs, with even-parity atoms cancelling as the label algebra requires. The output label is the XOR of its frontier atoms' labels, so a split at the output exists only if the frontier has at least two atoms with separable dependencies.

6.2 Results

Table 3 gives the result. Three findings of differing strength.

1. **Exact.** The output's linear frontier is a single atom. The backward walk terminated after one step with nothing unresolved, so the output wire is produced directly by a non-free gate with no XOR above it. The top-level decomposition Attack 4 requires does not exist. This is not statistical.
2. **Evidence.** The top non-free gate's operands each depend on inputs from all 64 residue classes with no mutually exclusive class, so they are not separably controllable at this resolution.
3. **Evidence.** Across the 638,658 non-free gates in the last 2^{21} gates of the circuit, none has separably controllable operands.

Claim 1. *Attack 4 does not apply to the deployed BN254 Groth16 verifier circuit.*

The reason is structural: the Groth16 verifier terminates in an equality check over the whole final-exponentiation result, so every wire near the output depends on essentially every input and there is nothing to vary independently.

6.3 Scope

The recorded tail is 2^{21} of 1.03×10^{10} gates, the last 0.02%. Finding 1 is exact and unconditional. Findings 2 and 3 hold at 64-class resolution, and two cones could in principle be disjoint while both touching all 64 classes; an exact analysis would carry a 3302-bit set per live wire, roughly 66 MB resident, and is future work. Separately, our non-free gate count of 2,714,835,821 differs from the upstream gate counter's 2,715,041,234 by 205,413, which we believe to be gates whose output wire is unreachable and which the upstream counter includes; this is unverified.

7 Discussion

7.1 From break to design requirement

The deployed circuit satisfies the structural condition, and no one chose that. It follows from Groth16's output convention, not from a decision. It is not obviously preserved under a change of proof system, a change of circuit compiler, or a change of output encoding. A hash-based verifier terminating in an XOR-tree accumulator, which is where the post-quantum direction for these protocols points, is a natural place for it to fail.

We therefore propose that evaluation-binding, and the structural condition that supports it, be stated and checked rather than inherited. The tool accompanying this paper decides the condition in one pass over the circuit.

7.2 Is $k = 128$ the right parameter here?

The 128-bit label length is inherited from two-party computation, where the garbler cannot choose the evaluator's input. BitVM-style protocols are not that setting whenever labels reach an evaluator without on-chain authentication. If Attack 4 applies to some future circuit, restoring 128-bit security requires $k = 256$, which doubles the garbled circuit. For the verifier measured here that is 40.5 GB becoming 81 GB, in a design where off-chain size is the binding constraint. The parameter deserves explicit justification in this setting rather than inheritance.

7.3 Relation to prior attacks

Futoransky, Larotonda and Barbara [5] broke an earlier RSA-based garbling proposal for this same application with a circuit containing two AND gates. Their attack is evaluator-side and exploits multiplicative structure in $\mathbb{Z}_N$, which half-gates with a correlation-robust hash does not have, so it does not transfer. The shape is instructive nonetheless: in both cases the claim is that a party cannot produce labels it should not be able to produce, the intuition was that structure made it hard, and the structure was exploitable.

8 Open problems

1. Prove or refute evaluation-binding for half-gates under circular correlation robustness [7], for circuits satisfying the structural condition. Attack 4 bounds what any such proof can achieve at $2^{k/2}$ in general.
2. Give a decision procedure for the structural condition that is exact rather than sketch-based, and cheap enough to run in continuous integration.

3. Characterise which circuit output conventions guarantee the condition by construction, so that it can be enforced by the compiler rather than audited after the fact.
4. Determine whether arithmetic garbling schemes ([4, 8]) admit an analogous garbler-side binding gap, and whether their algebraic structure makes it easier or harder to exploit.

Artifacts

All code and measurements are available at `https://github.com/<user>/garbler-binding`. The exploit is a single Rust file with one dependency and runs in about ten seconds. The circuit analysis requires one pass over the BN254 Groth16 verifier, roughly ten minutes on a laptop core.

Acknowledgements

This work was carried out with substantial assistance from Claude (Anthropic) for literature reconciliation, implementation and analysis.

References

[1] Ekrem Bal, Lukas Aumayr, Atacan Iyidogan, Giulia Scaffino, Hakan Karakus, Cengiz Eray Aslan, and Orfeas Stefanos Thyfronitis Litos. Clementine: A collateral-efficient, trust-minimized, and scalable bitcoin bridge, 2025.

[2] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In *ACM CCS*, 2012.

[3] BitVM Alliance. garbled-snark-verifier, 2026.

[4] Liam Eagen. Glock: Garbled locks for bitcoin, 2025. Cryptology ePrint Archive, Paper 2025/1485.

[5] Ariel Futoransky, Gabriel Larotonda, and Fadi Barbara. A note on the security of the BitVM3 garbling scheme, 2025. Cryptology ePrint Archive, Paper 2025/1291.

[6] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.

[7] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *IEEE S&P*, 2020. Cryptology ePrint Archive, Paper 2019/074.

[8] Nakul Khambhati, Anwesh Bhattacharya, and David Heath. Duty-free bits: Projectivizing garbling schemes, 2026. Cryptology ePrint Archive, Paper 2026/476.

[9] Nakul Khambhati, Mukesh Tiwari, Sapin Bajracharya, Manish Bista, Liam Eagen, Christian Lewe, and Aaron Feickert. Mosaic: Practical malicious security for garbled circuits on bitcoin, 2026. Cryptology ePrint Archive, Paper 2026/812.

[10] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In *ICALP*, 2008.

[11] Robin Linus. BitVM: Compute anything on bitcoin, 2023.

[12] Robin Linus, Lukas Aumayr, Alexei Zamyatin, Andrea Pelosi, Zeta Avarikioti, and Matteo Maffei. BitVM2: Bridging bitcoin to second layers, 2024.

[13] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12(1), 1999.

[14] Robin Linus Woll, Ioannis Alexopoulos, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and David Tse. BitVM3: Efficient bitcoin bridges via garbled circuits, 2026. Cryptology ePrint Archive, Paper 2026/933.

[15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT*, 2015. Cryptology ePrint Archive, Paper 2014/756.